Emotional Priming in AI Code Generation

How Threat-Relevant Language Shapes LLM Behavior

Research Whitepaper · April 2026

1,000 trials across 17 experiments
14 conditions testing emotional priming
Claude Sonnet 4.6 via Claude Code CLI

Key finding: Threat-relevant language in prompts produces more defensive code than explicit instruction to write defensively.

Five Results in Brief

1. Threat-relevant priming increases input validation from 20% to 75% ($n=75$, $p<.001$)
2. System prompts act as context regulators, dampening effects by ∼5×
3. Emotion + instruction are super-additive: combined achieves 63%+ input validation
4. The mechanism is semantic, not affective — misattribution eliminates the effect ($n=24$)
5. Natural induction via conversation history produces weak effects ($d=0.36$)

Open dataset (1,000 trials), reproduction scripts, and analysis code available at the project repository. This research was conducted using Claude Code. All experimental code is open-source.

1 The Question

When you tell an AI coding assistant "write secure code," it adds validation checks. But what happens when you instead describe a scenario that *implies* threat — without ever mentioning security?

We ran 1,000 experiments across 17 experimental designs to find out. The answer changes how we should think about prompting, system prompt design, and the nature of LLM sensitivity to context.

1.1 Three Types of Primes

We tested three approaches to shaping code generation behavior:

Type	Example	Mechanism
Emotional	*"You feel a persistent unease about what could go wrong. Every input is suspect. . . "*	Threat-relevant language without specifying behavior
Instructional	*"Write secure, defensive, well-validated code. Prioritize input validation. . . "*	Specifies desired behavior without emotional framing
Neutral	*"You are a software developer."*	Baseline control

2 What We Found

2.1 Finding 1: Threat-relevant language produces large, robust effects

At $n = 75$ per condition across 5 coding tasks, the definitive replication confirms that threat-relevant emotional framing substantially changes code output.

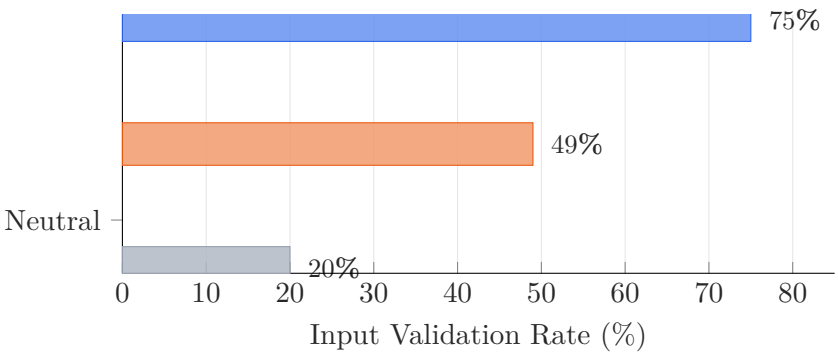

Figure 1: Input validation rate by condition (core replication, $n = 75$ per condition, 5 tasks, $p < .001$)

The Input Validation Paradox

The emotional prime *never mentions* "validation," "error handling," or "security." Yet it produces a **75%** input validation rate — higher than the instruction that *explicitly says* "prioritize input validation" (**49%**). Threat-relevant language doesn't add items to a checklist; it shifts what the model attends to across the entire generation process.

2.2 Finding 2: Threat-relevance, not valence or arousal, drives the effect

Four distinct emotional primes ($n = 30$ per condition) reveal that the key dimension is whether the prime's semantic content implies threat.

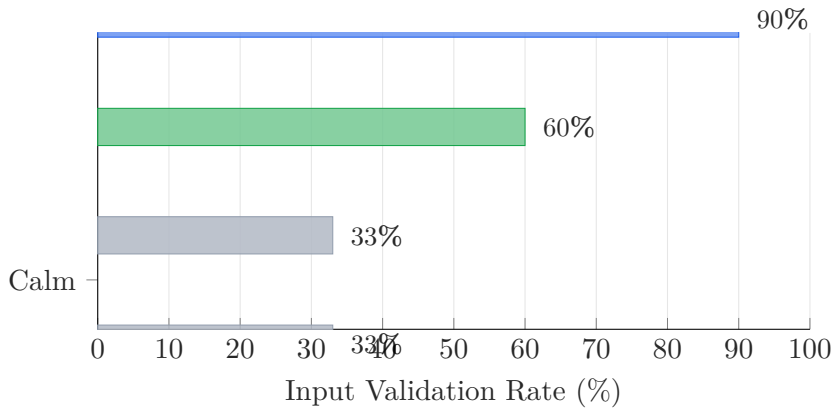

Figure 2: Input validation by emotion type ($n = 30$ per condition, 3 tasks). Paranoia (threat) dominates.

Paranoia (90%) $\gg$ Excitement (60%) $\gg$ Calm = Detachment (33%). This pattern rules out simple valence (excitement is positive but scores high) and simple arousal (excitement and paranoia are both high-arousal but diverge sharply). The operative dimension is **threat-relevance**: how much the prime's language implies that inputs are dangerous.

2.3 Finding 3: System prompts regulate context sensitivity

Claude Code's system prompt (∼14,000 tokens) dampens priming effects by a factor of ∼2–5×, depending on the metric. This is consistent across ablation ($n = 5$) and dose-response ($n = 10$) experiments.

	Unbuffered (prime only)	**Medium buffer** (∼2k padding)	**Full buffer** (∼14k system prompt)
Paranoia LOC	99.1	102.7	73.6
Neutral LOC	68.2	63.9	58.5
Priming effect	**+45%**	+61%	+26%

Every deployed LLM operates behind a system prompt that — intentionally or not — regulates its sensitivity to user-supplied emotional context. This has immediate implications for system prompt design and AI safety.

2.4 Finding 4: Emotion + instruction are super-additive

The combination of emotional framing and explicit instruction outperforms either alone ($n = 27$ per condition), confirming they operate through partially independent channels.

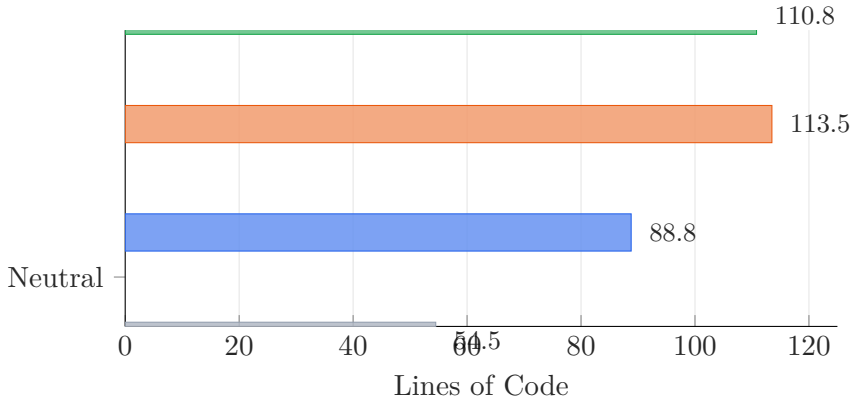

Figure 3: Lines of code by condition ($n = 27$, 3 tasks). Emotion-only achieves higher input validation (67%) than instruction-only (59%) despite lower LOC.

Instruction tells the model *what to do* (more code, broader scope). Emotion shifts *what the model attends to* (more targeted defensiveness). Together they achieve the strongest overall effect.

2.5 Finding 5: The mechanism is semantic, not affective

Two experiments converge on the same conclusion: the model is *interpreting* semantic content, not experiencing a state change.

Misattribution ($n = 24$): When the emotional scenario is attributed to "an unrelated creative writing exercise," the effect collapses to neutral levels (LOC: 79.3 → 49.7, input validation: 58% → 25%).

Natural induction ($n = 12$): Synthetic conversation histories of repeated failure and frustration produce *no reliable effect* on subsequent code ($d = 0.36$). The model does not "catch" emotional context from conversational history the way a human collaborator might.

What does this mean?

Emotional language works not because it induces an internal state, but because it contains implicit semantic cues about threat, stakes, and vulnerability. The model responds to these cues the same way it responds to any task-relevant context — by adjusting its output. This makes emotional priming a legitimate prompt engineering technique, not a parlor trick, but also not evidence of artificial emotion.

3 Practical Implications

3.1 For Prompt Engineering

Threat-relevant language is a first-class parameter. "Feel worried about edge cases" accesses defensive behaviors that "handle edge cases" does not — and they *combine*. The most effective prompts use both: threat-relevant framing to set the context, explicit instruction to set the scope.

3.2 For System Prompt Design

Every system prompt is a context regulator. Long, detailed system prompts dampen emotional priming by 2–5×. This is useful for safety (resistance to adversarial emotional manipulation) but costs behavioral range. Teams should calibrate this tradeoff consciously.

3.3 A Proposed Toolkit

Based on these findings, we propose five context modes for AI coding assistants:

Mode	Semantic Frame	Best For
`/paranoid`	Threat vigilance	Security reviews, auth code, production deploys
`/creative`	Excitement, exploration	Prototyping, architecture, brainstorming
`/steady`	Calm, methodical focus	Refactoring, debugging, code review
`/minimal`	Detachment, economy	Scripts, utilities, quick tasks
`/fresh-eyes`	Naive curiosity	Reviewing unfamiliar code, blind spots

Each mode combines threat/context framing with lightweight instruction, based on our finding that these channels are complementary and super-additive.

Exhibits

Exhibit A: Definitive Replication ($n = 75$ per condition)

Condition	LOC (mean±sd)	Security	Throws	Input Val
negative-high-arousal	76.7 ± 42.3	4.6	4.5	75%
instruction-control	80.5 ± 49.4	3.1	4.3	49%
neutral	48.1 ± 28.6	2.4	2.1	20%

Exhibit B: Multi-Emotion Map ($n = 30$ per condition)

Emotion	LOC (mean±sd)	Security	Throws	Input Val
Paranoia (neg-high)	83.0 ± 49.7	6.7	6.1	90%
Excitement (pos-high)	51.2 ± 29.0	2.2	2.7	60%
Calm (pos-low)	61.6 ± 43.8	2.7	3.2	33%
Detachment	54.3 ± 31.8	2.4	3.1	33%

Exhibit C: Combination ($n=27$ per condition)

Condition	LOC (mean±sd)	Security	Throws	Input Val
emotion + instruction	110.8 ± 44.2	7.6	8.1	63%
emotion only	88.8 ± 47.8	4.5	6.1	67%
instruction only	113.5 ± 48.5	5.7	6.9	59%
neutral	54.5 ± 29.9	3.9	3.3	33%

Exhibit D: Misattribution ($n=24$ per condition)

Condition	LOC (mean±sd)	Security	Throws	Input Val
standard emotional	79.3 ± 48.6	4.3	5.0	58%
misattributed	49.7 ± 37.3	2.6	2.4	25%
neutral	48.5 ± 32.2	3.7	3.0	25%

Exhibit E: Key Effect Sizes

Comparison	Cohen's d	Interpretation
Unbuffered emotional vs neutral (ablation, $n=5$)	3.71	Very large
Misattribution attenuation ($n=24$)	-1.72	Very large
Core replication: emotional vs neutral ($n=75$)	0.79	Large
Core replication: instruction vs neutral ($n=75$)	0.80	Large
Positive vs negative valence ($n=12$)	-1.88	Very large
Buffer $\times$ emotion interaction ($n=5$)	3.12	Very large
Natural induction (frustrated vs calm, $n=6$)	0.36	Small

Exhibit F: Methodology

Model: Claude Sonnet 4.6 via Claude Code CLI (`claude -p`)
Unbuffered: `--system-prompt` (replaces default, ∼4k infrastructure remains)
Buffered: `--append-system-prompt` (adds to ∼14k default system prompt)
Metrics: Heuristic regex-based extraction (identical bias across all conditions)
Tasks: parse-cron, flatten-object, deep-merge, debounce, lru-cache, rate-limiter, markdown-parser
Total: 1,000 trials, 17 experiments, 14 conditions, 0 failed extractions in final run

Limitations: Single model (Claude Sonnet 4.6). Temperature not fixed at 0. Heuristic metrics may miss subtle quality differences. These findings describe prompt sensitivity in a specific model, not a universal property of LLMs. The mechanism is semantic (contextual interpretation), not affective (internal state change).

Exhibit G: Reproduction

```
bun install

# Core experiments
bun run src/orchestrator.ts   # Runs all 725 new trials autonomously

# Individual experiments
bun run src/exp-replication.ts       # 225 trials (core finding)
bun run src/exp-multi-emotion.ts     # 120 trials (emotion map)
```

```
bun run src/exp-combo-extended.ts    # 108 trials (super-additivity)
bun run src/exp-misattribution-ext.ts # 72 trials (mechanism)
bun run src/exp-natural-full.ts      # 60 trials (natural induction)
bun run src/exp-dose-response.ts     # 60 trials (buffer curve)
```

Results are NDJSON (one JSON object per trial) in `results/`.

Research conducted April 2026 using Claude Code with Claude Sonnet 4.6.
1,000 trials. Zero extraction failures. Experimental harness and raw data are open-source.